

如何实现将一个程序设计语言的 源程序转换为四元组

递归下降语法分析方法

算术表达式转换为四元组

• 例: $3*(5+4)$

四元组为:

- $(+, 5, 4, T1)$
- $(*, 3, T1, T2)$

如何将算术表达式转换为四元组

- 1. 写出文法规则
- 2. 写出递归下降分析程序
- 3. 添加四元组生成代码段

递归下降分析法

解决步骤:

(1) 写出进行简单算术运算的文法

[为了简单起见,只做+ * 括号运算]

$$E \rightarrow E + T \mid T$$
$$T \rightarrow T * F \mid F$$
$$F \rightarrow (E) \mid n$$

(2) 由于采用递归下降分析法,因此要先用EBNF改造文法.

文法改造结果:

$$E \rightarrow T \{ +T \}$$
$$T \rightarrow F \{ *F \}$$
$$F \rightarrow (E) \mid n$$

(3) 写出递归下降分析程序.

规则： $E \rightarrow T \{ + T \}$ 的代码为：

```
void E()  
{  
    T();  
    while (token == '+')  
    {  
        match (token) ;  
        T();  
    } // while  
} // E
```

规则: $T \rightarrow F \{ * F \}$ 的代码为:

```
void T()  
{  
    F();  
    while (token == '*')  
    {  
        match (token);  
        F();  
    } //while  
} //T
```

规则： $F \rightarrow (E) \mid n$ 的递归动作：

```
void F()
```

```
{
```

```
    switch( token )
```

```
    {
```

```
        case '(':
```

```
            match('(') ;
```

```
            E();
```

```
            match(')') ;
```

```
            break;
```

```
        case number :
```

```
            match (n) ;
```

```
            break;
```

```
        default:
```

```
            error() ;
```

```
    } // switch
```

```
} // F
```


规则： $F \rightarrow (E) \mid n$ 的递归动作：

```
void F()  
{  
    switch( token )  
    {  
        case '(':  
            match('(');  
            E();  
            match(')');  
            break;  
        case number :  
            match (n);  
            // 产生一个操作数  
            break;  
        default:  
            error();  
    } // switch  
} // F
```

规则: $T \rightarrow F \{ * F \}$ 的代码为:

```
void T()
```

```
{
```

```
    F();
```

```
    while (token == '*')
```

```
    { match (token);
```

```
        F();
```

```
        // 产生一个新的临时变量
```

```
        // 产生了一个新临时变量后, 序号+1
```

```
        Gen(*,x,y,z); //生成一个四元组
```

```
    } //while
```

```
} //T
```

规则： $E \rightarrow T \{ + T \}$ 的代码为：

```
void E()
```

```
{
```

```
  T();
```

```
  while (token == '+')
```

```
  {
```

```
    match (token) ;
```

```
    T();
```

```
    // 产生一个新的临时变量
```

```
    // 产生了一个新临时变量后，序号+1
```

```
    Gen(+,x,y,z); //生成一个四元组
```

```
  } // while
```

```
} // E
```

main() 的安排

```
{  
    getToken();  
    E();  
}
```

- 例子: $3*(5+4)$

- 运行结果为:

- $(+, 5, 4, T1)$

- $(*, 3, T1, T2)$

赋值语句转换为四元组

例： $x := 3 * (5 + 4)$

四元组为：

$(+, 5, 4, T1)$
 $(*, 3, T1, T2)$
 $(:=, T2, , x)$

如何实现赋值语句转换为四元组

- 1. 写出文法规则
- 2. 写出递归下降分析程序
- 3. 添加四元组生成代码段

如何实现赋值语句转换为四元组

- $S \rightarrow id := E$

例: $x := 3 * (5 + 4)$

四元组为:

$(+, 5, 4, T1)$

$(*, 3, T1, T2)$

$(:=, T2, , x)$

```
void assign_stmt(void)
```

```
{
```

```
    // 为ID产生一个操作数
```

```
    match(ID);
```

```
    op=token;
```

```
    match(ASSIGN);
```

```
    x=exp();
```

```
    Gen(op,x,y,z); //生成一个四元组
```

```
}
```

Read和write语句转换为四元组

- read x;
- write y;

四元组为：

- (read, , , x)
- (write, , , y)

TINY 语言的语法规则列表

program $\rightarrow$ stmt-sequence

stmt-sequence $\rightarrow$ stmt-sequence ; statement | statement

statement $\rightarrow$ if-stmt | repeat-stmt | assign-stmt | read-stmt | write-stmt

if-stmt $\rightarrow$ if exp then stmt-sequence end

| if exp then stmt-sequence else stmt-sequence end

repeat-stmt $\rightarrow$ repeat stmt-sequence until exp

assign-stmt $\rightarrow$ identifier := exp

read-stmt $\rightarrow$ read identifier

write-stmt $\rightarrow$ write exp

exp $\rightarrow$ simple-exp comparison-op simple-exp | simple-exp

comparison-op $\rightarrow$ < | =

simple-exp $\rightarrow$ simple-exp addop term | term

addop $\rightarrow$ + | -

term $\rightarrow$ term mulop factor | factor

mulop $\rightarrow$ * | /

factor $\rightarrow$ (exp) | number | identifier

EBNF中TINY语言的文法

program $\rightarrow$ *stmt-sequence*

stmt-sequence $\rightarrow$ *statement* { **;** *statement* }

statement $\rightarrow$ *if-stmt* | *repeat-stmt* | *assign-stmt* | *read-stmt* | *write-stmt*

if-stmt $\rightarrow$ **if** *exp* **then** *stmt-sequence* [**else** *stmt-sequence*] **end**

repeat-stmt $\rightarrow$ **repeat** *stmt-sequence* **until** *exp*

assign-stmt $\rightarrow$ **identifier** **:=** *exp*

read-stmt $\rightarrow$ **read** **identifier**

write-stmt $\rightarrow$ **write** *exp*

exp $\rightarrow$ *simple-exp* [*comparison-op* *simple-exp*]

comparison-op $\rightarrow$ **<** | **=**

simple-exp $\rightarrow$ *term* { *addop* *term* }

addop $\rightarrow$ **+** | **-**

term $\rightarrow$ *factor* { *mulop* *factor* }

mulop $\rightarrow$ ***** | **/**

factor $\rightarrow$ (*exp*) | **number** | **identifier**

- Tiny样本语言：求输入值阶乘TINY语言程序

```
{ Sample program
  in TINY language -
  computes factorial
}
read x; { input an integer }
if 0 < x then { don't compute if x <= 0 }
  fact := 1;
  repeat
    fact := fact * x;
    x := x - 1
  until x = 0;
  write fact { output factorial of x }
end
```

• Tiny语言源代码

```
{ Sample program
  in TINY language--
  computes factorial
}
read x; { input an int
if 0 < x then { don't
  fact := 1;
  repeat
    fact := fact * x;
    x := x - 1
  until x = 0;
  write fact { output
end
```

```
(1)(rd, , , x)
(2)(j<, 0, x, ? )
(3)(j, , , ? )
(4)(:=, 1, _, fact)
(5) (*, fact, x, T1 )
(6)(:=, T1 , , fact )
(7) (-, x, 1 , T2 )
(8) (:=, T2 , , x )
(9)(j=, x , 0 , 11 )
(10)(j, , , 5 )
(11)(WR, , , fact )
```